

Group Assignment Work Plan and Task Split

DIC2026 - Assignment 3 · Deadline **30 June 2026, 23:59 (TUWEL)** · ~12 days · 5 people (P1-P5) · async, text-only → handoff workflow.

The assignment in brief

Build an **event-driven serverless app on MiniStack** that analyses customer reviews: preprocess text (tokenize, stop-word removal, lemmatize), **profanity check**, **sentiment analysis**, **count impolite reviews per customer**, and **ban a customer with more than 3** impolite reviews. Grading is about serverless design, not speed/accuracy.

Submit **one zip** named `<groupID>_DIC2026_Assignment_3.zip` containing:

- `report.pdf` — max 8 pages, 11pt, one column, ≥5 sections (Introduction, Problem Overview, Methodology, Results, Conclusions), **must include an architecture diagram**.
- `instructions.pdf` — how to run the app.
- `src/` — well-documented code + integration tests + any extra test reviews.

Three things that will bite us if ignored:

1. `reviews_devset.json` **is not in the folder** — all required results are computed on it. Find it first (likely from Assignment 1/2 or the LBD cluster).
2. **DynamoDB is effectively required** (per-customer counting/banning needs stored state; the brief mentions "table names" and DynamoDB events). Not optional.
3. **The full dataset run is the only heavy compute** — finish it by **Fri 26 June** so a busy cluster near the deadline can't sink us.

Requirement checklist

Source = `Assignment_3_Instructions.pdf` unless noted. Priority: **M** must, **S** should.

Requirement	Pri	Owner
≥3 Lambdas: preprocessing, profanity, sentiment	M	P2, P3
Chain starts when one review is added to an S3 bucket	M	P1, P2
Other invocations triggered only by S3/DynamoDB events (fan-out via EventBridge/SNS — see Tips PDF)	M	P1
Use <code>summary</code> , <code>reviewText</code> , <code>overall</code> for profanity and/or sentiment	M	P2, P3
All bucket/table names from SSM parameter store (no hardcoding)	M	P1, P2, P3
Preprocess: tokenize, stop-word removal, lemmatize (NLTK)	M	P2
Profanity check (<code>profanityfilter</code>) on the text fields	M	P2
Sentiment analysis pos/neu/neg (NLTK)	M	P3
Count impolite reviews per customer (DynamoDB)	M	P3
Ban when impolite count > 3, i.e. on the 4th	M	P3
One deploy script recreates ALL resources (MiniStack is ephemeral)	M	P1
Results on devset only: <code>#pos/#neu/#neg</code> , <code>#failed profanity</code> , banned users	M	P4
Integration tests for all 5 functionalities (extend <code>tests/test_integration.py</code>)	M	P4
<code>report.pdf</code> (5 sections + diagram, ≤8pg/11pt/1col) + <code>instructions.pdf</code>	M	P5
Ship extra/corner-case test reviews in <code>src/</code> (kept out of reported numbers)	S	P4

Requirement	Pri	Owner
Code documented + black-formatted	S	all
AI-usage doc if course requires it (not stated in these files — confirm on TUWEL)	S	P5

Who does what

Order: **P1 → (P2 || P3) → P4 → P5**. P2 and P3 build in parallel once P1's contract is ready. Each section below is self-contained.

P1 — Architecture & infrastructure

In short: design the function chain, build the one-command deploy script, define all SSM params + DynamoDB/S3 schema, find the dataset, draw the diagram. You unblock everyone.

- **Tasks:** Find `reviews_devset.json` and confirm its fields (incl. a customer/reviewer ID + `summary / reviewText / overall`). Design the chain: S3 input bucket → preprocessing → fan-out (EventBridge or SNS) → profanity + sentiment → DynamoDB → DynamoDB-event-triggered counter/ban. Write `deploy.sh` modeled on `assignment_3_tutorial/run.sh` (create buckets, DynamoDB table, SSM params, all Lambdas, S3 notifications, fan-out). Put all names in SSM (`/dic-reviews/...`). Draw the **architecture diagram** (style of `assignment_3_tutorial/architecture.png`). Write a 1-page `CONTRACT.md`: what each Lambda reads/writes, the JSON shape passed between stages, S3 key + DynamoDB schema, SSM names.
- **Reference:** `run.sh`, `lambdas/*/handler.py` (SSM pattern), `Tips_and_Tricks.pdf` (fan-out commands).
- **Output:** `src/deploy.sh`, SSM params, DynamoDB/S3 schema, `architecture.png`, `CONTRACT.md`, deployable empty Lambda skeletons that read names from SSM.
- **Depends on:** nothing — start now.
- **Hand off:** post that the contract + deploy script work → P2, P3 start.
- **Done when:** fresh MiniStack + `bash deploy.sh` creates everything with no error; skeletons read SSM.

P2 — Preprocessing & profanity Lambdas

In short: the entry Lambda (preprocessing) and the profanity Lambda.

- **Tasks:** preprocessing Lambda — triggered by a review added to the input S3 bucket; read `summary` + `reviewText` (keep `overall`); tokenize, remove stop words, lemmatize with NLTK; write normalized output per `CONTRACT.md`. Bundle NLTK data into the ZIP (≤ 250 MB unzipped). profanity Lambda — run `profanityfilter` over `summary` + `reviewText`; write an `impolite` flag + which field hit. Write a short methodology note (preprocessing + profanity) for the report.
- **Reference:** `assignment_3_tutorial/lambdas/resize/handler.py` for the S3-event + SSM pattern; P1's `CONTRACT.md`.
- **Files:** `src/lambdas/preprocessing/`, `src/lambdas/profanity/`. Don't edit `deploy.sh` — ask P1 if you need a new param/table.
- **Output:** two working Lambdas + `requirements.txt` + bundling steps; methodology note.
- **Depends on:** P1 contract + skeletons.
- **Hand off:** give P3 the preprocessing output schema + `impolite-flag` location.
- **Done when:** uploading one review triggers preprocessing; `impolite` test review is flagged, clean one is not; names only via SSM.

P3 — Sentiment, counting & ban Lambda

In short: the sentiment Lambda and the per-customer counting + ban logic.

- **Tasks:** sentiment Lambda — classify each review pos/neu/neg with NLTK (e.g. VADER) over `summary / reviewText`, and factor in `overall`. ban logic — triggered by a **DynamoDB event**; increment a customer's impolite count on each impolite review; set `banned=true` when count > 3 (so 3 → not banned, 4 → banned; check this edge case). Write a short methodology note (sentiment + counting/ban) for the report.
 - **Reference:** P1's `CONTRACT.md` + DynamoDB schema; P2's impolite-flag output.
 - **Files:** `src/lambdaas/sentiment/`, `src/lambdaas/ban/`. Don't edit `deploy.sh` — request changes from P1.
 - **Output:** sentiment Lambda + counter/ban Lambda; methodology note.
 - **Depends on:** P1 contract (sentiment can start immediately); P2's impolite flag (for counting).
 - **Hand off:** tell P4 where sentiment labels + ban state live in DynamoDB.
 - **Done when:** known-sentiment reviews classified correctly; 4 impolite reviews for one user → banned; all triggers are S3/DynamoDB events.
-

P4 — Dataset run, results & integration tests

In short: push the whole dataset through the chain, produce the 3 required numbers, write the tests. You own the heavy compute.

- **Tasks:** Build a batch loader that uploads every review in `reviews_devset.json` as a separate object into the input bucket and waits for the chain. **Aggregate over the devset only:** `#positive/` `#neutral/#negative`, `#reviews failing profanity`, `banned users` → save to `src/results/results.json`. Write integration tests (extend `tests/test_integration.py`) covering all 5 functionalities using small fixtures (not the whole devset). Create corner-case reviews (e.g. a user with 4+ impolite reviews) in `src/test_reviews/` — shipped, but excluded from the reported numbers.
 - **Files:** `src/tools/load_devset.py`, `src/tests/`, `src/results/`, `src/test_reviews/`. Don't fix Lambda internals — report bugs to P2/P3 as a failing test.
 - **Output:** batch loader, `results.json` + the 3 numbers, green pytest suite (5 functionalities), corner-case reviews.
 - **Depends on:** P2 + P3 Lambdas deployed.
 - **Hand off:** give P5 the result numbers + a passing test log.
 - **Done when:** `pytest` is green (5/5); results reproduce from a fresh deploy; numbers are devset-only.
 - **Timing:** finish the **full run by Fri 26 June** — do not leave it for the last days.
-

P5 — Report, instructions & submission

In short: write both PDFs, assemble the zip, run the final check. Single integrator.

- **Tasks:** Compile `report.pdf` from P1/P2/P3 notes + P4 results: 5 sections (Introduction, Problem Overview, Methodology, Results, Conclusions) + P1's architecture diagram; enforce **≤8 pages, 11pt, one column**. Write `instructions.pdf` with P1 (start `ministack`, `bash deploy.sh`, add a review, run `pytest`, reproduce results). Assemble `src/`; run `black --check`; confirm no hardcoded names. **Reproduce on a clean MiniStack** using only `instructions.pdf`. Confirm the AI-usage policy on TUWEL; add an AI-usage note if required. Build and verify `<groupId>_DIC2026_Assignment_3.zip`; submit via TUWEL.
 - **Output:** `report.pdf`, `instructions.pdf`, final zip, completed final check.
 - **Depends on:** notes from P1/P2/P3, results + tests from P4.
 - **Done when:** clean-machine reproduction works; report numbers match `results.json`; zip named correctly and uploaded before 30 June 23:59.
-

Deliberate overlaps: P4 validates P2/P3 by writing tests against their Lambdas (bugs go back as failing tests). P4 runs the dataset; **P5 independently reproduces** the results from a clean deploy. P5 writes the report; a second person checks it against the checklist above before submitting.

Timeline (18–30 June)

Weekends (20–21, 27–28) kept light. Keep it loose, but protect the 26 June run and the 30 June buffer.

Dates	Focus	Who
Thu 18 – Fri 19	P1: find dataset, design chain, <code>deploy.sh</code> , SSM, DynamoDB schema, diagram, contract	P1
Sat 20 – Tue 23	P2 + P3 build Lambdas in parallel; P4 writes loader + test scaffolding	P2, P3, P4
Wed 24 – Fri 26	Wire end-to-end; P4 runs tests + full dataset run (done by Fri 26)	P2, P3, P4
Sat 27 – Sun 28	P5 compiles <code>report.pdf</code> + <code>instructions.pdf</code> ; assemble <code>src/</code>	P5 (+P1)
Mon 29	Reproduce on clean MiniStack; final check; build zip; submit a first version	P5 + all
Tue 30	Buffer only — fixes, formatting, re-upload. No new compute.	all

Final check before submitting

- [] Fresh MiniStack + `bash src/deploy.sh` creates everything, no error.
- [] One review upload triggers the full chain; devset results regenerate to the same numbers.
- [] `pytest` green — 5 tests for the 5 functionalities.
- [] Report numbers match `results/results.json`; diagram shows all Lambdas, PaaS, and triggers.
- [] No hardcoded names (all via SSM); no broken paths in `instructions.pdf`.
- [] Files present: `report.pdf` (≤ 8 pg, 11pt, 1col), `instructions.pdf`, `src/` (lambdas, `deploy.sh`, tests, `test_reviews`, results).
- [] Nothing on 30 June needs the cluster.
- [] AI-usage doc included if required.
- [] Zip = `<groupID>_DIC2026_Assignment_3.zip`, uploaded via TUWEL before 30 June 23:59.